

Java EE Module 1

Introduction to Java EE

Study Notes

Q1. What is Java EE? Explain its architecture.

Java EE (Java Platform, Enterprise Edition) is a set of specifications and APIs built on top of Java SE that provides tools and frameworks for building large-scale, multi-tier, distributed, and web-based enterprise applications. It is now called **Jakarta EE** (since Oracle transferred it to Eclipse Foundation).

In simple terms: Java EE gives you ready-made building blocks (APIs) so you don't have to write everything from scratch when building complex business applications.

Architecture Three-Tier Model

Java EE follows a **multi-tier (layered) architecture**:

Tier	What it is	Components
Client Tier	The user's browser or app	HTML, JavaScript, JSP pages
Middle / Web Tier	Handles requests & logic	Servlets, JSP, JSF, EJB
Data / EIS Tier	Stores data	Database, JDBC, JPA

How it works:

- The client sends a request (e.g., fills a form).
- The **Web Tier** (Servlet/JSP) receives it, processes business logic (possibly via EJBs).
- The **Data Tier** (JDBC/JPA) interacts with the database.
- A response is sent back to the client.

Key containers in Java EE:

- **Web Container** runs Servlets and JSPs (e.g., Apache Tomcat).
- **EJB Container** runs business components (Enterprise Java Beans).
- **Application Client Container** for standalone client apps.

Q2. Differentiate between Java SE and Java EE.

Java SE (Standard Edition) is the core Java platform it gives you the basic language features, data structures, I/O, networking, etc. **Java EE** is built *on top of* Java SE and adds enterprise-specific APIs for building web and distributed applications.

Feature	Java SE	Java EE
Full name	Standard Edition	Enterprise Edition
Purpose	General-purpose apps, desktop apps	Large-scale web & enterprise apps
APIs	Core: Collections, I/O, Threads	+ Servlet, JSP, JDBC, EJB, JPA, JMS
Target	Individual developers, small apps	Businesses, distributed systems
Runtime	JVM	JVM + Application Server (e.g., GlassFish, JBoss)
Complexity	Simpler	More complex, more features
Example	Calculator, file utility	Online banking portal, e-commerce site

Analogy: Java SE is the foundation (bricks and cement). Java EE is the full blueprint for constructing a skyscraper using those bricks.

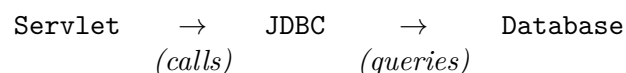
Q3. What is the role of JDBC in web applications?

JDBC (Java Database Connectivity) is a Java API that allows a Java application to connect to and interact with a relational database (like MySQL, Oracle, PostgreSQL). It is the bridge between your Java code and the database.

Role in web applications:

- **Connect** to the database using a driver and connection URL.
- **Execute SQL queries** INSERT, SELECT, UPDATE, DELETE.
- **Retrieve results** via `ResultSet` and pass them to JSP/Servlet.
- **Manage transactions** commit or rollback operations.

Typical JDBC flow in a web app:



Key JDBC classes:

- `DriverManager` loads the DB driver, creates connection.
- `Connection` represents an open connection to the DB.
- `Statement` / `PreparedStatement` executes SQL.
- `ResultSet` holds the query results.

Example use: A user logs in → Servlet calls JDBC → JDBC runs `SELECT * FROM users WHERE username=?` → Result is checked → User is authenticated.

Q4. What is the role of JSP in web applications?

JSP (JavaServer Pages) is a server-side technology that lets you embed Java code directly inside HTML pages. It is used to create **dynamic web pages** whose content changes based on data, user input, or logic.

Role in web applications:

- **Presentation Layer** Responsible for displaying data to the user (View in MVC).
- Combines **HTML** (structure) with **Java** (dynamic content).
- Displays data passed from a Servlet (e.g., user name, product list).
- Supports **JSTL** (JSP Standard Tag Library) for cleaner code without raw Java.

How JSP works:

1. Browser sends a request.
2. Server converts the `.jsp` file into a Servlet (auto).
3. Servlet runs, generates HTML.
4. HTML response is sent to browser.

JSP elements:

Element	Syntax	Purpose
Scriptlet	<code><% code %></code>	Run Java code
Expression	<code><%= value %></code>	Print a value
Directive	<code><%@ page ... %></code>	Page settings
Declaration	<code><%! method %></code>	Declare methods/variables

Q5. What is the role of Servlets in web applications?

A **Servlet** is a Java class that runs on the server and handles HTTP requests from clients (browsers). It acts as the **controller** in web applications it receives a request, processes it, and decides what response to send (usually by forwarding to a JSP).

Role in web applications:

- **Request Handling** Intercepts HTTP GET/POST requests.
- **Business Logic** Processes user input, validates data.
- **Data Coordination** Calls JDBC to query DB, passes data to JSP.
- **Session Management** Tracks users using `HttpSession`.
- **Response Generation** Forwards to JSP or directly writes HTML output.

Servlet lifecycle:

1. `init()` Called once when servlet loads.
2. `service()` Called for each request; routes to `doGet()` or `doPost()`.
3. `destroy()` Called once when servlet is removed.

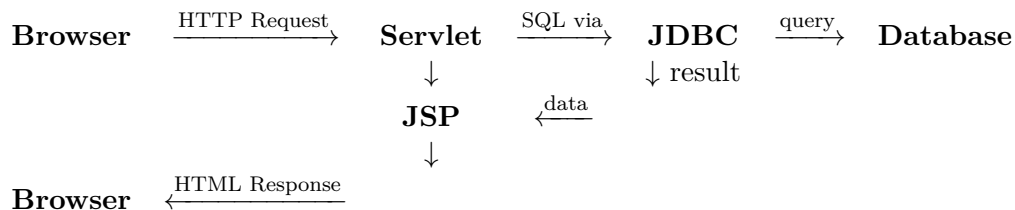
Example: User submits login form → `LoginServlet.doPost()` runs → Validates using JDBC → Forwards to `home.jsp` on success or `error.jsp` on failure.

Q6. Compare the roles of JDBC, JSP, and Servlet in a web application.

These three technologies work **together** in a layered web application. Each handles a different responsibility, following the classic **MVC (Model-View-Controller)** pattern.

Aspect	Servlet	JSP	JDBC
MVC Role	Controller	View	Model (data access)
Purpose	Handle requests, control flow	Display dynamic HTML to user	Connect & query the database
Written in	Pure Java	HTML + embedded Java	Pure Java
Focus	Logic & routing	Presentation	Data persistence
Interacts with	JSP, JDBC	User's browser	Database
Example task	Process login form	Show welcome page	Check credentials in DB
File extension	.java	.jsp	.java (DAO class)

How they collaborate a complete flow:



Memory trick:

- **Servlet** = *Brain* makes decisions.
- **JSP** = *Face* shows things to the user.
- **JDBC** = *Hands* fetches and stores data.